# Project 2 – Acme Analytics Linux Platform

**Day 5 · Phase 2 complete, Phase 3 started: snapshot rollback, job control, systemd timer and socket**

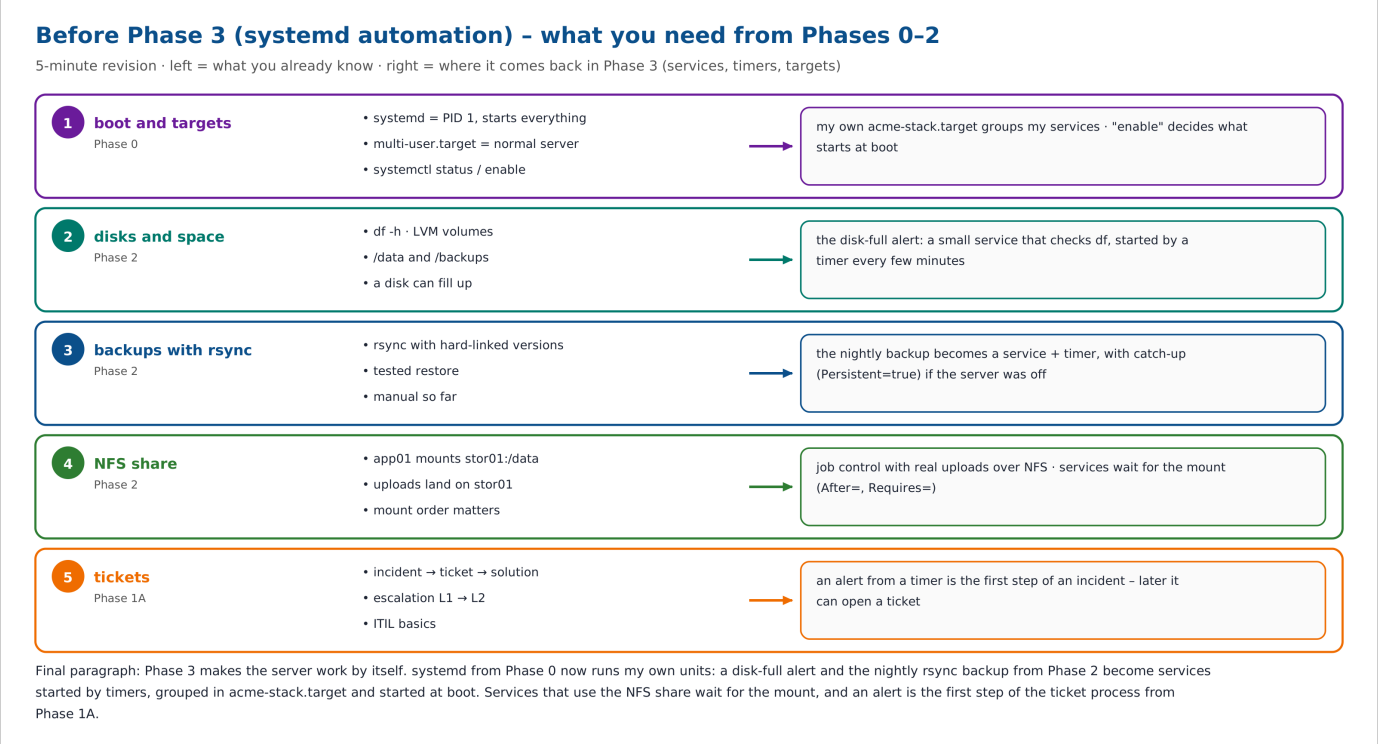
Muhammad Bilal Javed · 20 October 2026 · study copy

| | |
|---|---|
| **Project** | Acme Analytics – hardened, observable and recoverable Linux platform |
| **Phase** | 2 – Storage, sharing and recovery · 3 – Automation |
| **Hosts worked on** | gw01 (access test), stor01 (LVM snapshot, NFS server), app01 (NFS client) |
| **Status** | Phase 2 ✅ – Steps 2.1–2.4 done; VMware snapshots `30-storage-*` · Phase 3 🟡 – Step 3.1 job control ✅ · Step 3.2 disk-full alert as a systemd service + timer ✅ · Step 3.3 socket activation ✅ |
| **Author** | Muhammad Bilal Javed |

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 4.

## Before Phase 3 – quick revision (Phases 0–2)

💡 **5 minutes:** what I already know from Phases 0–2 and where it comes back in Phase 3.



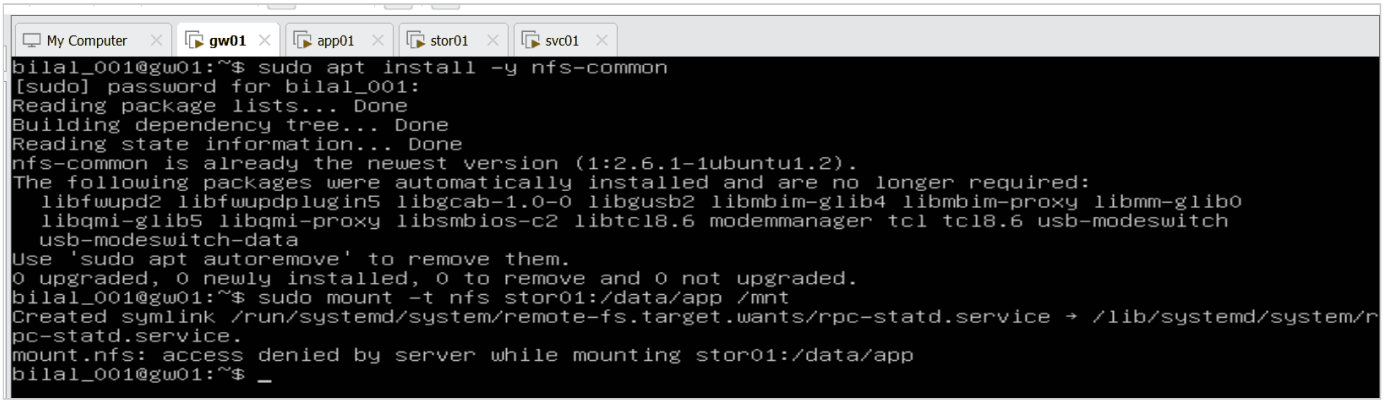

*Before Phase 3 – quick revision*

## 1. Break test – reachable is not allowed (Step 2.2)

💡 **In plain words:** gw01 has stor01 in `/etc/hosts` and can ping it – but it is not listed in stor01's `/etc/exports`. Can it use the share anyway?

```
# on gw01
sudo apt install -y nfs-common                  # NFS client tools (already installed)
sudo mount -t nfs stor01:/data/app /mnt         # /mnt = standard empty folder for temporary mounts
```

📸 **Evidence – `access denied by server`:**




*access denied*

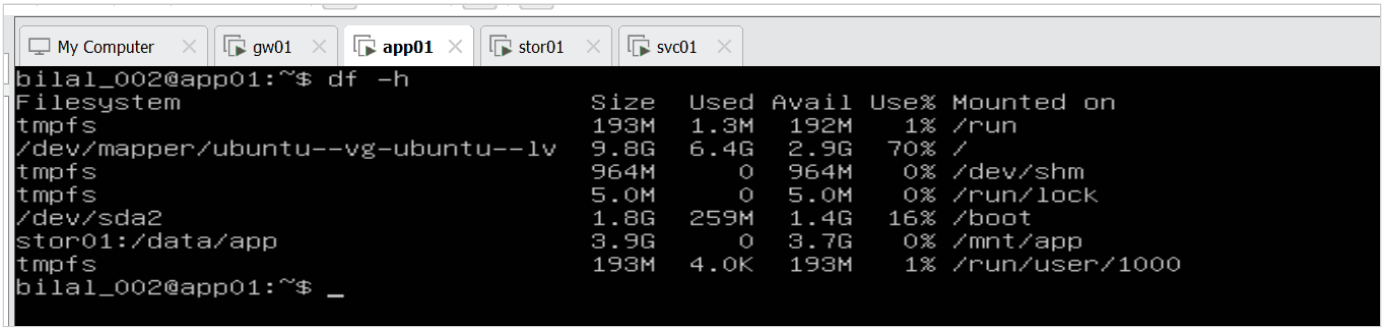
**Lesson:** being able to reach a server is not permission to use its shares. Only the IPs listed in `/etc/exports` (here only app01, 192.168.60.10) may mount `/data/app`.

## 2. Where the app's data really lives

```
app01: /mnt/app  ═ NFS ═▶  stor01: /data/app  ─inside─▶  /data  ─mounted from─▶  lvdata (logical volume)
                                                  /backups ─mounted from─▶  lvbackups
```

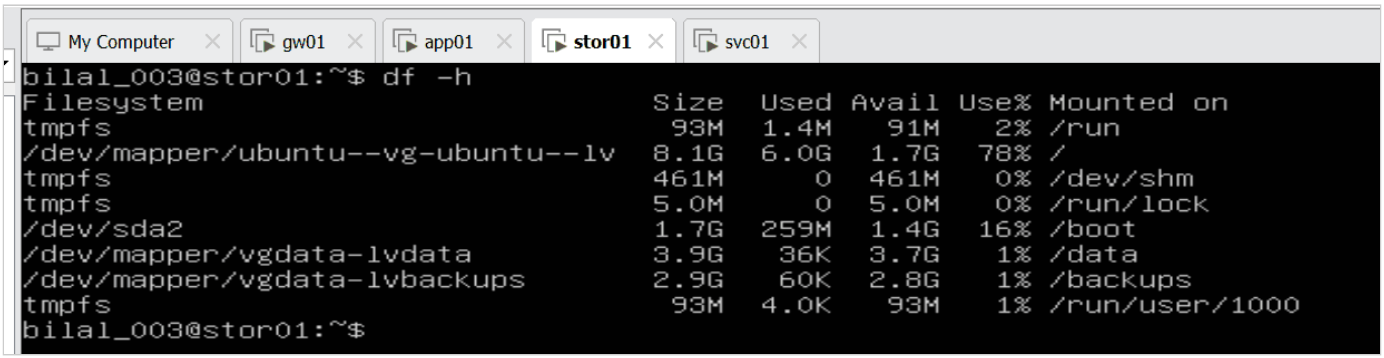
- `/mnt/app` is a folder on **app01**; `/data/app` is a folder on **stor01**. Over NFS, everything done in `/mnt/app` really happens in `/data/app`.
- On stor01 the logical volume **`lvdata`** is mounted on `/data` – so every file in `/data/app` is stored on `lvdata`. **The work happens on `lvdata`; the backups are on another volume, `lvbackups`.**

📸 **Evidence – app01: `stor01:/data/app` on `/mnt/app`:**




*app01 df*

📸 **Evidence – stor01: `lvdata` on `/data`, `lvbackups` on `/backups`:**




*stor01 df*

## 3. LVM snapshot – the undo button (Step 2.4)

💡 **The story:** a risky change is coming to `/data` (an update, a cleanup). A mistake made from app01 lands directly on `lvdata`, and the rsync mirror – which keeps only the current state – would copy it at its next run. The dated versions still hold good files, but restoring file by file is slow. So **before** the change: take a snapshot – a frozen state of the whole volume – and roll back in seconds if it goes wrong.

## 3.1 Take the snapshot, make the “mistakes”

```
# on app01 — release the share first (see the issue below)
sudo umount /mnt/app

# on stor01
ls /data/app                                              # lab.txt new.txt test.txt
sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata        # -s snapshot of lvdata; 1 GB room for changes
sudo lvs
echo "bad change" | sudo tee /data/app/oops.txt           # mistake 1
sudo rm /data/app/test.txt                                # mistake 2
ls /data/app                                              # lab.txt new.txt oops.txt
```

| `lvs` shows | Meaning |
|---|---|
| `lvdata  owi-aos---` | **o** = origin: a snapshot is watching this volume |
| `snap1  swi-a-s---  1.00g  Origin lvdata` | **s** = snapshot of lvdata, 1 GB room for changes |
| `Data% 0.01` | How full the snapshot’s room is – at 100 % it becomes invalid |

The mistakes were made on stor01 for simplicity; from app01 through `/mnt/app` the effect is identical – it is the same file on `lvdata`, and the snapshot protects the volume, whoever made the change.

📸 **Evidence:**

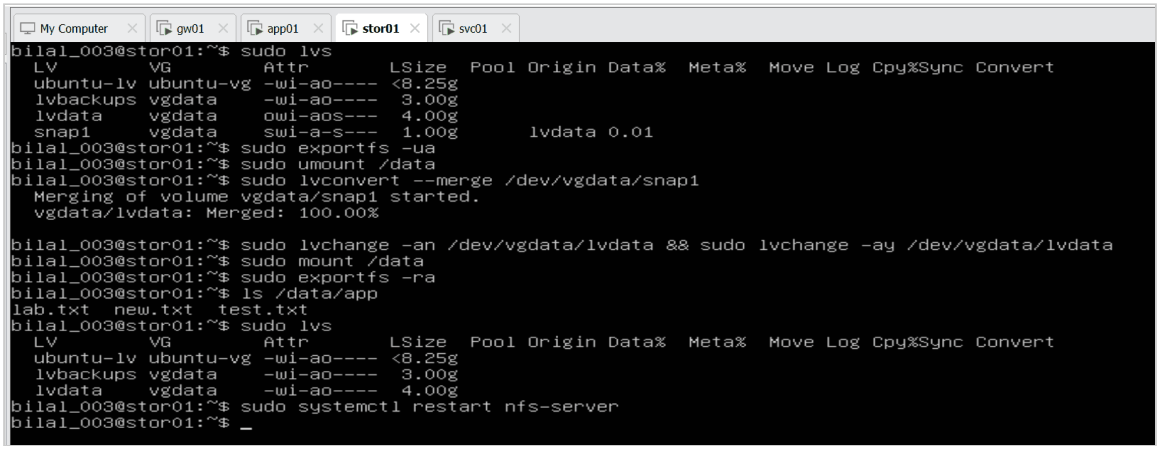
```
bilal_003@stor01:~$ ls /data/app
lab.txt  new.txt  test.txt
bilal_003@stor01:~$ sudo lvcreate -s -n snap1 -L 1G /dev/vgdata/lvdata
[sudo] password for bilal_003:
  Logical volume "snap1" created.
bilal_003@stor01:~$ sudo lvs
  LV        VG         Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  ubuntu-lv ubuntu-vg -wi-ao---- <8.25g
  lvbackups vgdata    -wi-ao----  3.00g
  lvdata    vgdata    owi-aos---  4.00g
  snap1     vgdata    swi-a-s---  1.00g      lvdata 0.01
bilal_003@stor01:~$ echo "bad change" | sudo tee /data/app/oops.txt
bad change
bilal_003@stor01:~$ sudo rm /data/app/test.txt
bilal_003@stor01:~$ ls /data/app
lab.txt  new.txt  oops.txt
bilal_003@stor01:~$ _
```

*snapshot taken*

## 3.2 Roll back

```
# on stor01
sudo exportfs -ua                                         # stop sharing /data/app
sudo umount /data                                         # nobody may use the volume during the merge
sudo lvconvert --merge /dev/vgdata/snap1                  # back to the frozen state
sudo lvchange -an /dev/vgdata/lvdata && sudo lvchange -ay /dev/vgdata/lvdata   # deactivate / activate
sudo mount /data
sudo exportfs -ra                                         # share again
ls /data/app                                              # lab.txt new.txt test.txt — oops.txt gone
sudo lvs                                                  # snap1 gone — merged into lvdata
```

📸 **Evidence – `Merged: 100.00%`; test.txt back, oops.txt gone; snap1 gone:**

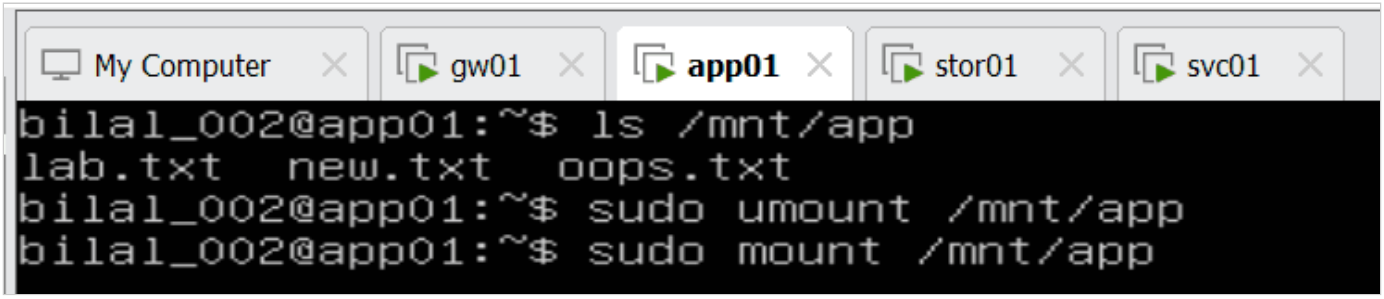



*rollback*

### 3.3 Issue – app01 hangs after the rollback (stale NFS mount)

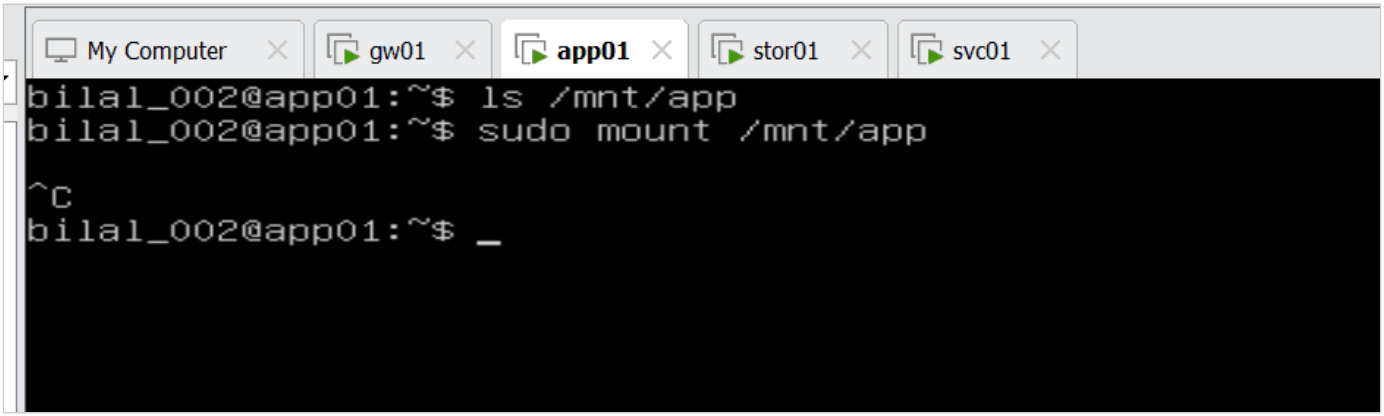
| | |
|---|---|
| **Seen** | After the rollback, `mount /mnt/app` on app01 hung and had to be stopped with Ctrl+C. |
| **Cause** | app01 still had the share mounted **during** the rollback – `ls /mnt/app` still showed `oops.txt`; it was unmounted only afterwards. stor01 pulled the volume away under a connected client, which was left with an old, invalid connection (a **stale** mount). |
| **Fix** | stor01: `sudo systemctl restart nfs-server` · app01: `sudo umount -f -l /mnt/app` (force, lazy), `showmount -e stor01`, `sudo mount -a` |
| **Lesson** | Before rolling back a volume shared over NFS, **unmount every client first** – the client lets go, then the server. |

📸 **Evidence – app01 still mounted during the rollback (oops.txt visible):**



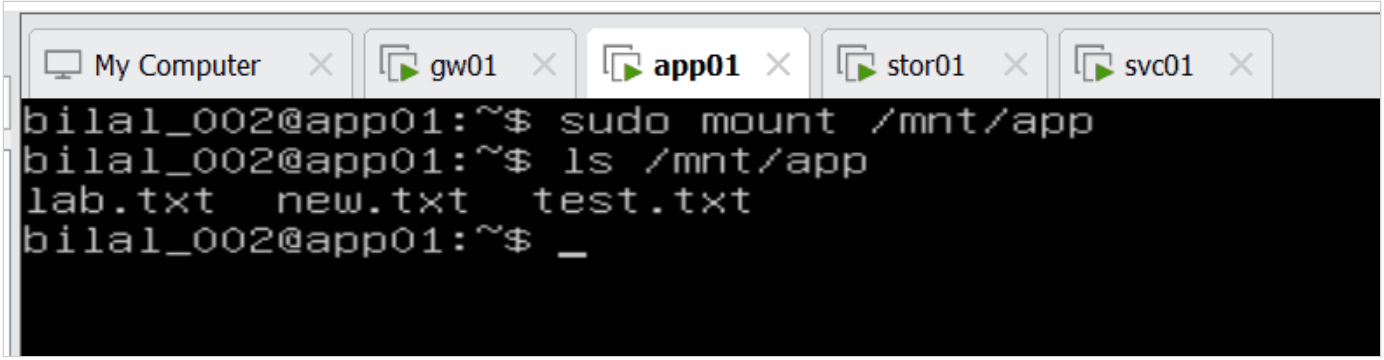

*still mounted*

📸 **Evidence – the mount hangs:**




*hang*

📸 **Evidence – after the fix, app01 sees the rolled-back files:**




*fixed*

## 4. Snapshot vs backup

| | LVM snapshot | Backup (versions in `/backups`) |
|---|---|---|
| What | A **frozen state of the whole volume** at the moment it was taken | **Separate copies** of the files |
| How | Keeps only the old version of blocks that change afterwards (copy-on-write), in its small room | Full files (unchanged ones hard-linked) |
| Where | **Same disks** as `lvdata` | Separate volume `lvbackups` – ideally another server |
| Restore | The **whole volume at once**, in seconds | Files **one by one**, from any day |
| Survives a reboot | Yes | Yes |
| Lost when | Disks die, its room fills to 100 %, merged or removed | Deleted, or the disks die – hence copies off-site |

**Snapshot = quick undo, short-term, same disks. Backup = recovery, long-term, separate volume.**

## 5. VMware snapshots

| VM | Snapshot | Description |
|---|---|---|
| gw01 | `30-storage-gw01` | End of Phase 2; NFS client tools for the access test |
| app01 | `30-storage-app01` | NFS client: stor01:/data/app on /mnt/app (fstab, `_netdev,nofail`) |
| stor01 | `30-storage-stor01` | LVM vgdata (lvdata /data 4 GB, lvbackups / backups 3 GB), NFS server, tar/rsync backups v1–v3 |
| svc01 | `30-storage-svc01` | GLPI; restore ticket closed |

### Key understanding (in my own words)

> On app01, `/mnt/app` is a folder. On stor01, `/data/app` is a folder. They are connected over NFS, so everything done in `/mnt/app` on app01 really happens in `/data/app` on stor01. On stor01, the logical volume `lvdata` is mounted on `/data`, so every file in `/data`, including `/data/app`, is stored on `lvdata`. The work happens on `lvdata`; the backups are stored on another logical volume, `lvbackups`, mounted on `/backups`.
>
> If someone on app01 makes a wrong or risky change in `/mnt/app`, the mistake lands directly on `lvdata`. The rsync mirror only keeps the current state, so at its next run it would copy the mistake too. A backup is a set of separate copies of the files on `lvbackups`: from it I can restore files one by one, from any day, but that is slow for a big mistake.
>
> So before a risky change I take an **LVM snapshot**: a frozen state of the whole `lvdata` volume, taken at the moment I create it. If the change goes wrong, I roll back the whole volume at once. To roll back, nobody may be using the volume: first app01 unmounts `/mnt/app`, then stor01 stops sharing (`exportfs -ua`) and unmounts `/data`. I learned this the hard way – app01 was still mounted during my rollback, was left with a stale mount and hung, until I restarted the NFS server and force-unmounted the client.
>
> Both survive a reboot, but a snapshot lives on the same disks and becomes invalid if its room fills up: **snapshot = quick undo, short-term, same disks; backup = recovery, long-term, separate volume.** And gw01 showed that being able to reach a server is not permission to use its shares.

## 6. Phase 3 starts – automation (Step 3.1: job control)

💡 **The story:** so far every command ran by hand, and long commands blocked the terminal. Acme's customers upload large files from **app01**; through NFS they land in `/data/app/uploads` on **stor01**, where they must be backed up. Long jobs like these should **not block my terminal**, **not slow down the app**, and **not die when my connection drops**.

```
app01: /mnt/app/uploads  ═ NFS ═▶  stor01: /data/app/uploads (lvdata)  ─backup─▶  /backups (lvbackups)
       "customers upload"                                                  nohup tar from an SSH session
```

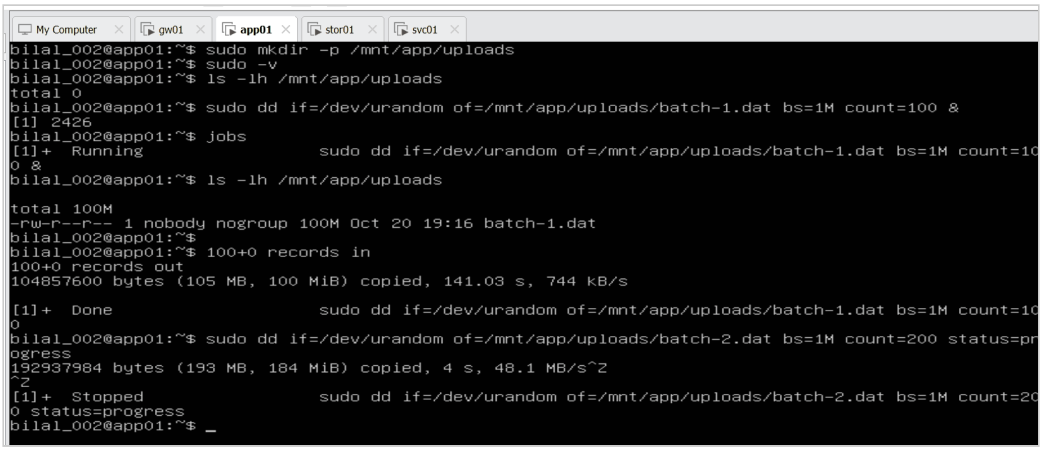
## 6.1 Customer uploads on app01 – foreground, background, pause, priority, kill

```
        # app01
        sudo rm -rf /mnt/app/uploads && sudo mkdir -p /mnt/app/uploads   # clean start (really /data/app/uploads on
stor01)
        sudo -v                                                          # cache the password — background sudo jobs
cannot ask for it

        # batch-1: in the background
        sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=100 &
        jobs                                                             # [1]+ Running — the prompt is free
        ls -lh /mnt/app/uploads                                          # keep working while it uploads

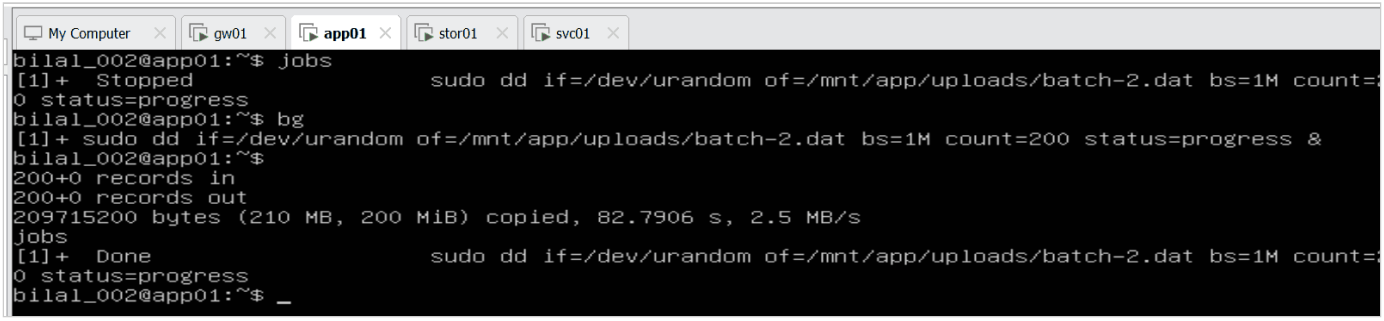
        # batch-2: forgot the & — pause and send to the background
        sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-2.dat bs=1M count=200 status=progress
        #   Ctrl+Z → [1]+ Stopped
        jobs; bg; jobs                                                   # Stopped → Running → Done
```

📸 **Evidence – batch-1 in the background (`Running` → `Done`), batch-2 paused with Ctrl+Z (`Stopped`):**




*bg and ctrl+z*

📸 **Evidence – `bg`: it continues in the background and finishes:**




*bg done*

### Issue – Ctrl+Z and Ctrl+C seemed to do nothing

| | |
|---|---|
| **Seen** | During an upload to `/mnt/app`, `^Z ^C ^Z …` appeared on the screen but the job did not react – until, after a while, `[1]+ Stopped`. |
| **Cause** | dd was writing to a **network disk**. While a process waits for NFS to confirm a write it is in state **D – uninterruptible sleep** and cannot react to signals until that write finishes. |
| **Check** | `ps -o pid,stat,cmd -C dd` – STAT `D` = waiting on the disk |
| **Lesson** | A slow or hanging NFS server can freeze programs on the client – the same root as yesterday's stale mount. dd's speed (575 kB/s) also counts the paused minutes; the real speed was about 9 MB/s. |

📸 **Evidence:**

```
bilal_002@app01:~$ sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=100
^Z


^C



^C^Z^Z^Z^Z^Z^Z



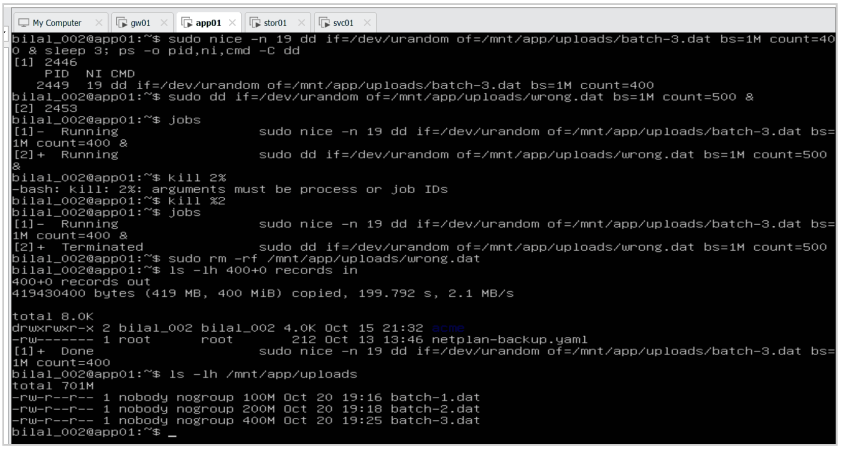
[1]+  Stopped                 sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=10
0
bilal_002@app01:~$ jobs
[1]+  Stopped                 sudo dd if=/dev/urandom of=/mnt/app/uploads/batch-1.dat bs=1M count=10
0
bilal_002@app01:~$ _
```

*ctrl+z waits*

```
        # batch-3: low priority — start dd and check its nice value in the same line
        sudo nice -n 19 dd if=/dev/urandom of=/mnt/app/uploads/batch-3.dat bs=1M count=400 & sleep 3; ps -o pid,ni,cmd -
C dd

        # wrong.dat: started by mistake — stop only that job
        sudo dd if=/dev/urandom of=/mnt/app/uploads/wrong.dat bs=1M count=500 &
        jobs                                # [1] batch-3 Running, [2] wrong.dat Running
        kill %2                             # [2] Terminated — batch-3 keeps running
        sudo rm -f /mnt/app/uploads/wrong.dat
        ls -lh /mnt/app/uploads             # batch-1 100M, batch-2 200M, batch-3 400M
```

**📸 Evidence – `NI 19`; two jobs at once, only `%2` killed; the three batches, owned by `nobody` (root_squash):**




*nice kill result*

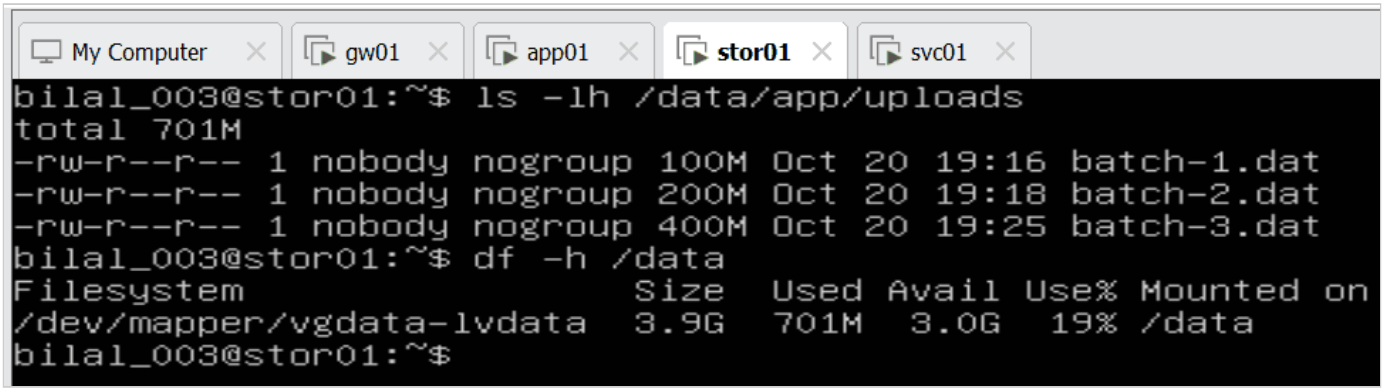
| Tool | Meaning | Real use |
|---|---|---|
| & | Start in the background | A long upload or backup without blocking the terminal |
| Ctrl+Z → bg / fg | Pause, continue in background / foreground | Rescue a job started without & |
| jobs | List my jobs with numbers | |
| kill %2 / kill PID | Stop by job number (same terminal) / by process ID (anywhere) | A job started by mistake |
| nice -n 19 / renice -n 19 -p PID | Priority -20 (most important) … 19 (politest), at start / while running | A daytime backup that does not slow the app |
| nohup … & | Keep running after logout | A long job over an SSH connection that may drop |

&/fg/bg decide **where** a job runs (does it block the terminal); nice/renice decide **how important** it is when the CPU is busy – all jobs still run at the same time. For disk-heavy jobs `ionice -c3` is the disk equivalent.

## 6.2 On stor01 – the uploads arrived; a backup that survives a logout

```
        # stor01
        ls -lh /data/app/uploads          # the three batches, 701 MB
        df -h /data                       # 19 % used
```

**📸 Evidence:**

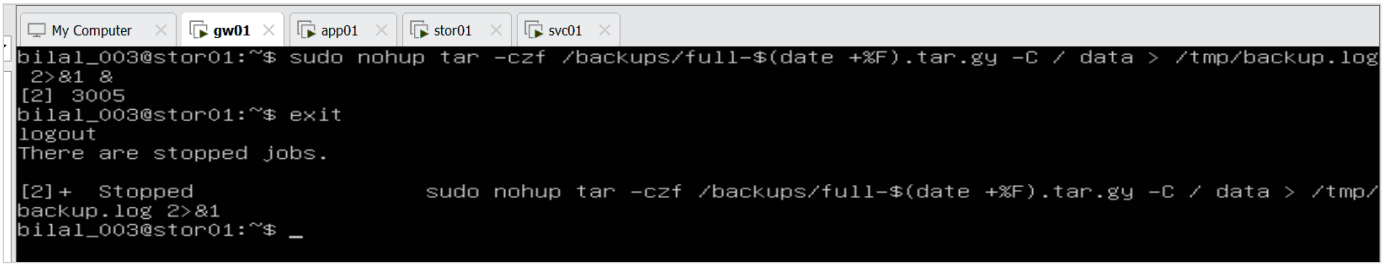



*uploads arrived*

### Issue – the background backup went “Stopped”

| | |
|---|---|
| **Seen** | The second `sudo nohup tar … &` showed `[2]+ Stopped`, and `exit` answered *There are stopped jobs* instead of logging out. |
| **Cause** | sudo wanted a password. A background job **cannot read from the keyboard**, so Linux paused it. Bash refuses the first `exit` while a job is stopped. |
| **Lesson** | Run `sudo -v` in the foreground **before** a background sudo job, and check `jobs` before `exit`. (The archive name also had a typo, `.tar.gy` – removed.) |

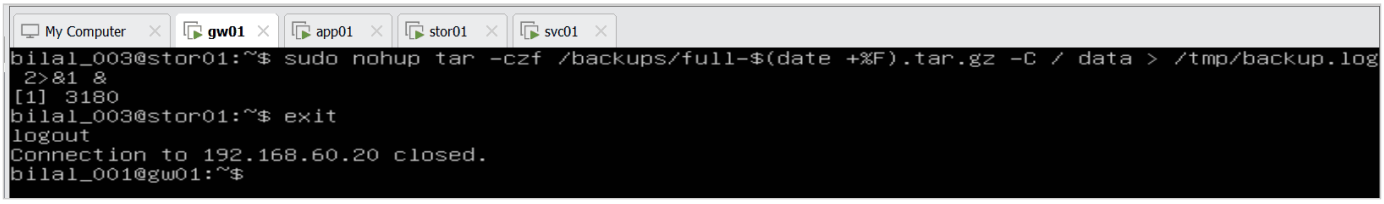
📸 **Evidence:**




*stopped*

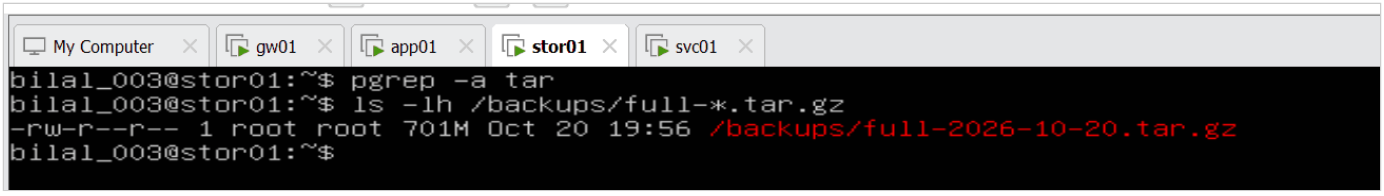
```
# gw01 — plays "my laptop"
ssh stor01
sudo -v
sudo nohup tar -czf /backups/full-$(date +%F).tar.gz -C / data > /tmp/backup.log 2>&1 &
exit                                    # really logged out: "Connection to 192.168.60.20 closed."
ssh stor01
pgrep -a tar                            # nothing — finished
ls -lh /backups/full-*.tar.gz           # 701M, finished at 19:56 — after I had left
sudo rm -f /backups/full-*.tar.gz       # free the space again
```

📸 **Evidence – started, logged out:**




*nohup logout*

📸 **Evidence – the archive was finished while I was gone:**




*finished after logout*

The archive is as big as the data (701 MB): random files – like real photos, videos and zipped uploads – hardly compress. Text and logs shrink a lot. That matters when planning backup space.

### Key understanding (in my own words)

Acme's customers upload files from app01 through `/mnt/app`, which NFS stores in `/data/app/uploads` on stor01. Long jobs like these uploads and backups should not block my terminal or slow down the app.

**Foreground and background:** any command can run in the foreground or the background. In the foreground, the terminal is busy until the command finishes. With `&` it runs in the background, and I get the prompt back at once. If I forget the `&`, `Ctrl+Z` pauses the job and `bg` lets it continue in the background; `fg` brings it back. `jobs` lists my jobs with their numbers. A wrong job is stopped with `kill %number` (job number, same terminal) or `kill PID` (process ID, from anywhere). On a network disk (NFS), a job waiting for a write reacts to Ctrl+Z or kill only after that write finishes.

**Priority:** `nice` and `renice` set a job's priority from -20 (most important) to 19 (politest). All jobs run at the same time, but when the CPU is busy, lower numbers get more CPU time. `nice` sets it at the start; `renice` changes it while the job runs. That is how a backup can run during the day without slowing down the app.

**Surviving a logout:** a job started from an SSH session is normally killed when the connection closes. With `nohup … &` it keeps running on the server after I log out. I started a 700 MB backup on stor01 from gw01, logged out, came back, and the archive had been finished while I was gone. Two lessons: run `sudo -v` first, because a background job cannot answer a password prompt (it becomes "Stopped"); and check `jobs` before `exit`, because Bash will not log out while a job is stopped. The archive was as big as the data (701 MB), because random files like customer uploads hardly compress.

## 7. Step 3.2 – disk-full alert: a systemd service and timer

💡 **The story:** files uploaded on app01 – today by me as a simulation, later by customers once the app is published through gw01 (Phase 7) – land in `/mnt/app`, which is really `/data` on stor01 (`lvdata`). If `/data` fills up, uploads fail and the app breaks, often before anyone in IT notices. In Phase 2 I can grow `/data` with LVM – but only if someone knows it is filling up. So, like a phone warning "storage almost full", stor01 must **watch its own disk and warn by itself** – no human, no manual command.

```
disk-alert.timer ──every 5 min──► disk-alert.service ──runs──► disk-alert.sh ──if too full──► journal (system
log)
     WHEN                              HOW                           WHAT
```

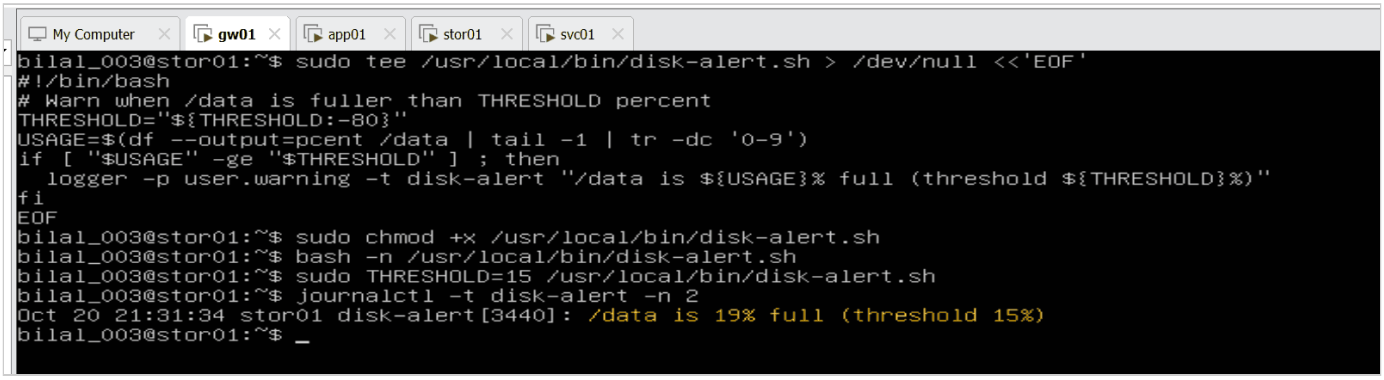
All on **stor01** – the disk to watch lives there.

### 7.1 The script – WHAT to check

```
sudo tee /usr/local/bin/disk-alert.sh > /dev/null <<'EOF'
#!/bin/bash
# Warn when /data is fuller than THRESHOLD percent
THRESHOLD="${THRESHOLD:-80}"
USAGE=$(df --output=pcent /data | tail -1 | tr -dc '0-9')
if [ "$USAGE" -ge "$THRESHOLD" ]; then
  logger -p user.warning -t disk-alert "/data is ${USAGE}% full (threshold ${THRESHOLD}%)"
fi
EOF
sudo chmod +x /usr/local/bin/disk-alert.sh
bash -n /usr/local/bin/disk-alert.sh               # syntax check without running – no output = OK
sudo THRESHOLD=15 /usr/local/bin/disk-alert.sh     # one manual run with a low test limit
journalctl -t disk-alert -n 2
```

| Line | Meaning |
|---|---|
| `#!/bin/bash` | Run this file with Bash |
| `THRESHOLD="${THRESHOLD:-80}"` | Use the given limit – or 80 % if none is given |
| `df --output=pcent /data \| tail -1 \| tr -dc '0-9'` | How full `/data` is, as a plain number: `tr -dc '0-9'` deletes everything except digits, so 19% → 19 |
| `if [ "$USAGE" -ge "$THRESHOLD" ]` | If usage is **g**reater or **e**qual to the limit … (`[` is a command – spaces inside `[ ]` are required) |
| `logger -p user.warning -t disk-alert "…"` | … write a **warning** to the system log, tagged `disk-alert` |

📸 **Evidence – syntax OK; manual run: `/data is 19% full (threshold 15%)`:**




*script manual run*

**Lesson:** my first version had two typing mistakes (a ) inside the quotes, no spaces inside `[ ]`). `bash -n` checks a script **before** it runs – use it every time.

## 7.2 The service (HOW) and the timer (WHEN)

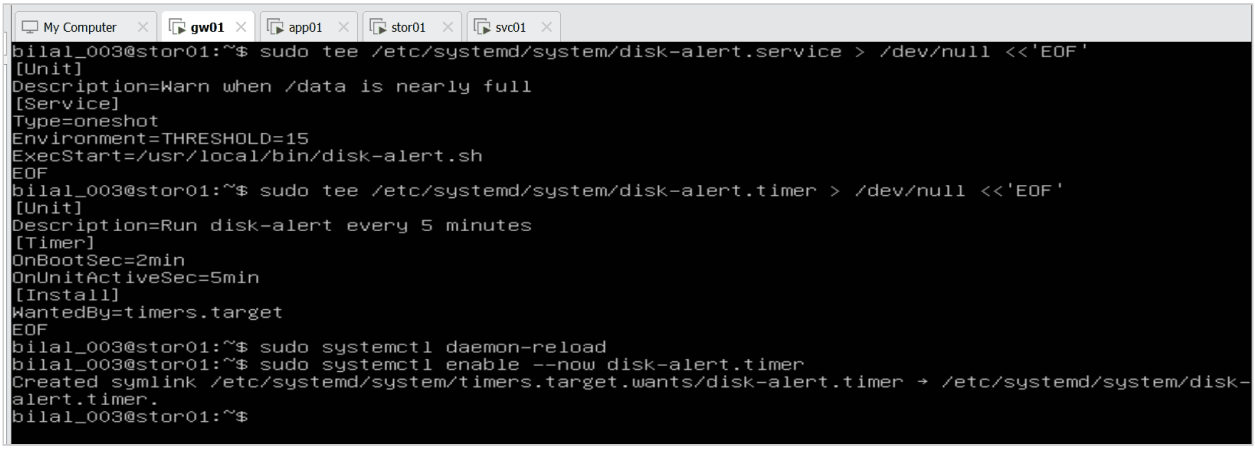
```
sudo tee /etc/systemd/system/disk-alert.service > /dev/null <<'EOF'
[Unit]
Description=Warn when /data is nearly full
[Service]
Type=oneshot
Environment=THRESHOLD=15
ExecStart=/usr/local/bin/disk-alert.sh
EOF

sudo tee /etc/systemd/system/disk-alert.timer > /dev/null <<'EOF'
[Unit]
Description=Run disk-alert every 5 minutes
[Timer]
OnBootSec=2min
OnUnitActiveSec=5min
[Install]
WantedBy=timers.target
EOF

sudo systemctl daemon-reload                    # systemd reads the new files
sudo systemctl enable --now disk-alert.timer    # start now + at every boot
systemctl list-timers | grep disk-alert
systemctl status disk-alert.service --no-pager
```

| Setting | Meaning |
|---|---|
| `Type=oneshot` | Run once and finish – the service does not stay running |
| `Environment=THRESHOLD=15` | Test limit – my uploads put `/data` at 19 %, so the alert fires now |
| `OnBootSec=2min` · `OnUnitActiveSec=5min` | First run 2 min after boot, then every 5 min |
| `WantedBy=timers.target` + `enable` | Start the timer automatically at every boot |

📸 **Evidence – units created, timer enabled:**




*service timer*

📸 **Evidence – `list-timers`: NEXT in 4 min, LAST 48 s ago; `status`:** TriggeredBy: disk-alert.timer**, `status=0/SUCCESS`, then `inactive (dead)` until the next run:**

```
bilal_003@stor01:~$ systemctl list-timers | grep disk-alert
Tue 2026-10-20 22:24:36 UTC 4min 11s left Tue 2026-10-20 22:19:36 UTC 48s ago       disk-alert.timer
                disk-alert.service
bilal_003@stor01:~$ systemctl status disk-alert.service --no-pager
● disk-alert.service - Warn when /data is nearly full
     Loaded: loaded (/etc/systemd/system/disk-alert.service; static)
     Active: inactive (dead) since Tue 2026-10-20 22:19:36 UTC; 1min 30s ago
TriggeredBy: ● disk-alert.timer
    Process: 3571 ExecStart=/usr/local/bin/disk-alert.sh (code=exited, status=0/SUCCESS)
   Main PID: 3571 (code=exited, status=0/SUCCESS)
        CPU: 85ms

Oct 20 22:19:36 stor01 systemd[1]: Starting Warn when /data is nearly full...
Oct 20 22:19:36 stor01 disk-alert[3576]: /data is 19% full (threshold 15%)
Oct 20 22:19:36 stor01 systemd[1]: disk-alert.service: Deactivated successfully.
Oct 20 22:19:36 stor01 systemd[1]: Finished Warn when /data is nearly full.
bilal_003@stor01:~$ _
```

*triggered by timer*

## 7.3 The real limit – silent when fine, loud when full

```
sudo sed -i 's/THRESHOLD=15/THRESHOLD=80/' /etc/systemd/system/disk-alert.service
sudo systemctl daemon-reload                        # always after changing a unit file
sudo systemctl start disk-alert.service             # 19 % < 80 % → no new line: no news is good news
journalctl -t disk-alert -n 3                       # 22:51, 22:56, 23:07 – runs by the timer, by themselves
sudo fallocate -l 2500M /data/app/uploads/huge-upload.dat    # simulate a huge upload – reserves 2.5 GB instantly
df -h /data                                         # 86 %
sudo systemctl start disk-alert.service
journalctl -t disk-alert -n 3                       # "/data is 86% full (threshold 80%)"
sudo rm /data/app/uploads/huge-upload.dat
df -h /data                                         # back to 19 %
```

📸 **Evidence – limit 80 %, silent at 19 %; the timer's own runs; the "huge upload" takes `/data` to 86 %:**

```
My Computer | gw01 | app01 | stor01 | svc01
bilal_003@stor01:~$ journalctl -t disk-alert -n 5
Oct 20 21:31:34 stor01 disk-alert[3440]: /data is 19% full (threshold 15%)
Oct 20 22:19:36 stor01 disk-alert[3576]: /data is 19% full (threshold 15%)
bilal_003@stor01:~$ sudo sed -i 's/THRESHOLD=15/THRESHOLD=80/' /etc/systemd/system/disk-alert.servic
e
[sudo] password for bilal_003:
bilal_003@stor01:~$ grep THRESHOLD /etc/systemd/system/disk-alert.service
Environment=THRESHOLD=80
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl start disk-alert.service
bilal_003@stor01:~$ journalctl -t disk-alert -n 5
Oct 20 22:51:24 stor01 disk-alert[3652]: /data is 19% full (threshold 15%)
Oct 20 22:56:27 stor01 disk-alert[3661]: /data is 19% full (threshold 15%)
Oct 20 23:07:06 stor01 disk-alert[3671]: /data is 19% full (threshold 15%)
bilal_003@stor01:~$ sudo fallocate -l 2500M /data/app/uploads/huge-upload.dat
bilal_003@stor01:~$ df -h /data
Filesystem                Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvdata  3.9G  3.2G  546M  86% /data
bilal_003@stor01:~$
```

*silent then 86*

📸 **Evidence – the alert fires: `/data is 86% full (threshold 80%)`; after clean-up 19 %:**

```
My Computer | gw01 | app01 | stor01 | svc01
bilal_003@stor01:~$ sudo systemctl start disk-alert.service
bilal_003@stor01:~$ journalctl -t disk-alert -n 3
Oct 20 22:56:27 stor01 disk-alert[3661]: /data is 19% full (threshold 15%)
Oct 20 23:07:06 stor01 disk-alert[3671]: /data is 19% full (threshold 15%)
Oct 20 23:13:37 stor01 disk-alert[3753]: /data is 86% full (threshold 80%)
bilal_003@stor01:~$ journalctl -t disk-alert -n 4
Oct 20 22:51:24 stor01 disk-alert[3652]: /data is 19% full (threshold 15%)
Oct 20 22:56:27 stor01 disk-alert[3661]: /data is 19% full (threshold 15%)
Oct 20 23:07:06 stor01 disk-alert[3671]: /data is 19% full (threshold 15%)
Oct 20 23:13:37 stor01 disk-alert[3753]: /data is 86% full (threshold 80%)
bilal_003@stor01:~$ journalctl -t disk-alert -n 5
Oct 20 22:46:06 stor01 disk-alert[3637]: /data is 19% full (threshold 15%)
Oct 20 22:51:24 stor01 disk-alert[3652]: /data is 19% full (threshold 15%)
Oct 20 22:56:27 stor01 disk-alert[3661]: /data is 19% full (threshold 15%)
Oct 20 23:07:06 stor01 disk-alert[3671]: /data is 19% full (threshold 15%)
Oct 20 23:13:37 stor01 disk-alert[3753]: /data is 86% full (threshold 80%)
bilal_003@stor01:~$ df -h /data
Filesystem                Size  Used Avail Use% Mounted on
/dev/mapper/vgdata-lvdata  3.9G  701M  3.0G  19% /data
bilal_003@stor01:~$
```

*alert 86*

| Time (UTC) | Journal | Meaning |
|---|---|---|
| 21:31 | 19 % (threshold 15 %) | Manual run of the script |
| 22:19 → 23:07 | 19 % (threshold 15 %) | **The timer**, every ~5 min, by itself (gaps = VM paused) |
| ~23:08 | – | Limit 80 %: the disk is fine, the script stays silent |
| 23:13 | **86 % (threshold 80 %)** | The simulated huge upload crossed the real limit |

### Key understanding (in my own words)

Files uploaded on app01 (today by me as a simulation, later by customers once the app is published through gw01) go into `/mnt/app`, which is really `/data` on stor01, the `lvdata` volume. If `/data` fills up, new uploads fail and the app breaks, often before anyone in IT notices. In Phase 2 I could grow `/data` with LVM, but only if someone knows it is filling up. So, like a phone that warns "storage almost full", stor01 should **watch its own disk and warn us by itself**, without a human running a command. The script `/usr/local/bin/disk-alert.sh` reads how full `/data` is with `df`, keeps only the number with `tr -dc '0-9'` (so `19%` becomes `19`), and compares it with a threshold: 80 % normally, 15 % for my test. If the usage is greater than or equal to the threshold, `logger` writes a **warning to the system log**, tagged `disk-alert`, which I read with `journalctl -t disk-alert`. I checked the script with `bash -n` and ran it once by hand.

Then I turned it into automation: `disk-alert.service` says *how* to run it (`Type=oneshot` – run once and finish), and `disk-alert.timer` says *when* – 2 minutes after boot and then every 5 minutes. `systemctl enable --now` started the timer and made it survive reboots; `status` showed *TriggeredBy: disk-alert.timer* and *SUCCESS*. The timer wakes the service, the service runs the script, and the script writes a warning only if `/data` is too full. With the test limit, warnings appeared every few minutes by themselves. With the real limit of 80 % (`sed` + `daemon-reload`) the script stayed silent at 19 %; a simulated 2.5 GB upload (`fallocate`) pushed `/data` to 86 %, and the alert *"/data is 86% full (threshold 80%)"* appeared. stor01 now watches its own disk, and the warning comes early enough to grow the volume or clean up.

## 8. Step 3.3 – socket activation: start a service only when someone connects

💡 **The story:** a rarely used service does not need to run all day and use memory. Instead, **systemd itself listens on the port** and starts the program only when a client connects – then it ends again. Real systems use this: `ssh.socket` (newer Ubuntu), `docker.socket`, `cups.socket` (printing).

**What a unit is.** Everything systemd manages is a **unit** – a small text file in `/etc/systemd/system/`. The ending says what kind:

| Unit type | Says | In this project |
|---|---|---|
| `.service` | **how** to run a program | `disk-alert.service`, `hello@.service` |
| `.timer` | **when** to start a service | `disk-alert.timer` |
| `.socket` | **which port** to listen on – start the service on connect | `hello.socket` |
| `.target` | a **group** of units | `multi-user.target` (Day 1), `acme-stack.target` (next) |

```
client connects to :9999 ──► hello.socket (systemd listens)
                                │ same name "hello" → template hello@.service
                                ▼
                     one copy starts → echo → output back to the client → ends
```

All on **stor01**.

```
sudo tee /etc/systemd/system/hello.socket > /dev/null <<'EOF'
[Socket]
ListenStream=9999
Accept=yes
[Install]
WantedBy=sockets.target
EOF
sudo tee /etc/systemd/system/hello@.service > /dev/null <<'EOF'
[Service]
ExecStart=/bin/sh -c 'echo "hello from stor01 — started on demand"'
StandardOutput=socket
EOF
sudo systemd-analyze verify /etc/systemd/system/hello@.service   # check the unit before testing
sudo systemctl daemon-reload
sudo systemctl enable --now hello.socket
ss -tlnp | grep 9999                                             # LISTEN *:9999 — systemd holds the port
nc localhost 9999                                                # → hello from stor01 — started on demand
```

| Setting | Meaning |
|---|---|
| `ListenStream=9999` | Listen on TCP port 9999 |
| `Accept=yes` | Each connection gets its own short-lived copy of the template `hello@.service` |
| `@` in `hello@.service` | **Template** – one instance per connection |
| `StandardOutput=socket` | The program's output goes back through the connection to the client |

📸 **Evidence – the socket unit:**

```
bilal_003@stor01:~$ sudo tee /etc/systemd/system/hello.socket > /dev/null <<'EOF'
[Socket]
ListenStream=9999
Accept=yes
[Install]
WantedBy=sockets.target
EOF
```

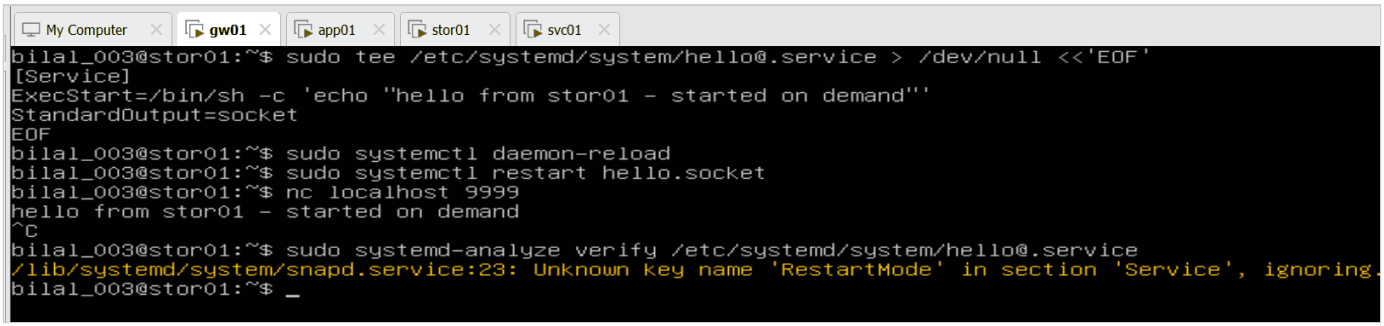
*socket unit*

📸 **Evidence – enabled; `LISTEN *:9999`:**

```
bilal_003@stor01:~$ sudo systemctl daemon-reload
bilal_003@stor01:~$ sudo systemctl enable --now hello.socket
Created symlink /etc/systemd/system/sockets.target.wants/hello.socket → /etc/systemd/system/hello.so
cket.
bilal_003@stor01:~$ ss -tlpn | grep 9999
LISTEN 0      4096               *:9999             *:*
bilal_003@stor01:~$ nc localhost 9999
```

*listening*

📸 **Evidence – the greeting on connect; `systemd-analyze verify` shows no error for my file (the yellow line is about Ubuntu's own `snapd.service`):**




*on demand*

**Lessons from my mistakes:** the service was first named `hello.service` (with `Accept=yes` it must be the template `hello@.service`), the section was misspelled (`[Servuce]`), and `ExecStart` lacked the leading `/` – each time `nc` just waited. `ExecStart` needs a full path, `cat` the unit once, and `systemd-analyze verify` catches such mistakes before testing.

### Key understanding (in my own words)

> In systemd, everything it manages is a **unit**: a small text file in `/etc/systemd/system/` that describes one thing, and the ending of the name says what kind. A **`.service`** says *how* to run a program, a **`.timer`** says *when* to start a service, a **`.socket`** says *which network port* to listen on, and a **`.target`** is a *group* of units, like `multi-user.target` (server mode). After creating or changing a unit, `systemctl daemon-reload` makes systemd read it; `enable` starts it at every boot and `--now` also starts it immediately. In Step 3.2 I used a service plus a timer for the disk alert. In Step 3.3 I used a **socket**: a rarely used service does not need to run all day, so systemd itself listens on the port (`hello.socket`, port 9999) and starts the program **only when someone connects**. The socket and the service are linked by their name (`hello`). With `Accept=yes`, each connection gets its own short-lived copy of the template `hello@.service` (the `@` means template), and `StandardOutput=socket` sends the program's output straight back through the connection to the client. `nc localhost 9999` returned *"hello from stor01 – started on demand"*; before and after, nothing was running. Real systems use the same idea, for example `ssh.socket`, `docker.socket` and `cups.socket`. My mistakes taught me three checks: the template needs the `@`, `ExecStart` needs a full path starting with `/`, and `systemd-analyze verify` checks a unit file before testing – like `bash -n` for scripts.

## 9. Next steps

Continued in Day 6.

1. **A real-world socket-activation example** (instead of a greeting) – then Step 3.4: the `acme-stack.target` grouping the Acme units.
2. **Step 3.5:** cron vs systemd timer, and the **nightly backup** of `/data/app` running by itself at 02:00 (`Persistent=true` catches up after downtime).
3. Phase 2 tickets in GLPI: LVM build, grow /data, fstab UUID typo, NFS share, stale NFS mount after rollback.